

Architecture Decision Record

ADR-002 : Stratégie de découpage en modules

Date : 09/03/2026

Statut : **Proposé**

Auteurs : Teddy SMAILI, Axel WOEL LALA, Tatiana ABI SALEH, Martin BONNAND

1. Contexte

QuickCommerce est actuellement organisé en monolithe : une seule application Java (Tomcat 8.5) contenant tout le code métier (Catalogue, Commande, Paiement, Stock, Livraison, Compte client).

Cette architecture pose plusieurs problèmes :

- Déploiement lourd : toute modification nécessite de redémarrer l'application complète
- Code spaghetti : dépendances enchevêtrées entre les différentes fonctionnalités
- Équipe bloquée : conflits Git fréquents sur le monolithe
- Scalabilité impossible : on ne peut pas scaler sélectivement une fonctionnalité
- Tests lents : la suite de tests complète prend 45 minutes

À la suite de l'analyse de la séance 2, nous avons décidé de migrer vers une infrastructure cloud résiliente (AWS Multi-AZ). Cette migration est l'occasion de repenser le découpage logiciel.

Contraintes :

- Équipe de 3 personnes avec peu d'expérience DevOps/microservices
- Migration progressive sans interruption de service
- Maintien de la vélocité de développement

2. Options envisagées

Trois approches ont été évaluées pour restructurer QuickCommerce.

Structure suggérée :

Option A : Monolithe modulaire

Une seule application déployée, mais le code est organisé en modules internes bien séparés (Catalogue, Commande, Paiement, etc.). Les modules communiquent via des interfaces Java claires, sans accès direct aux données des autres. Déploiement unique, complexité opérationnelle faible.

Option B : SOA (Service-Oriented Architecture)

3 à 5 services indépendants déployés séparément (ex : Catalogue, Commande/Paiement, Livraison). Communication via API REST ou bus de messages. Compromis entre autonomie des équipes et complexité opérationnelle modérée.

Option C : Microservices complets

10+ microservices indépendants, chacun avec sa propre base de données. Maximum d'autonomie et de scalabilité, mais nécessite une infrastructure avancée (Kubernetes, service mesh, monitoring distribué) et une forte expérience DevOps.

3. Décision

Option A - Monolithe modulaire.

4. Justification

Le monolithe modulaire est le choix le plus adapté aux contraintes actuelles de QuickCommerce. L'équipe de 3 personnes dispose de peu d'expérience en DevOps et en gestion de services distribués : adopter une SOA ou des microservices introduirait une complexité opérationnelle disproportionnée par rapport aux bénéfices attendus. Le monolithe modulaire permet de respecter les principes de faible couplage et forte cohésion en découpant le code en 7 modules métier (Catalogue, Panier, Commande, Paiement, Stock, Livraison, Compte client), tout en conservant un seul déploiement simple à gérer. Cette approche permet également une migration progressive depuis le monolithe actuel, sans interrompre le service. Enfin, elle laisse la porte ouverte à une évolution vers des services indépendants dans le futur, une fois que l'équipe aura acquis les compétences nécessaires.

Points à aborder :

- L'équipe a-t-elle les compétences pour gérer cette architecture ?
- La migration peut-elle se faire progressivement ?
- Cette architecture respecte-t-elle la règle faible couplage + forte cohésion ?
- Quelle complexité opérationnelle ?

5. Conséquences

Avantages :

- Déploiement simple : une seule application à gérer et surveiller
- Faible complexité opérationnelle, adaptée à une équipe de 3 personnes sans expérience DevOps
- Migration progressive possible depuis le monolithe actuel, sans interruption de service

Inconvénients / Risques :

- Scalabilité limitée : impossible de scaler un seul module indépendamment
- Déploiement complet à chaque modification, même mineure
- Risque de régression vers le code spaghetti si les frontières entre modules ne sont pas respectées

6. Modules identifiés

Compte Client, Catalogue, Panier, Commande, Paiement, Stock, Livraison
